

The Concept of Attacks

To effectively understand attacks on the different services, we should look at how these services can be attacked. This is a concept that is applied to future projects. As an example, we can think of the concept of building a house. Most homes are built this way, and it is a concept that is applied all over the world. The material used or the type of design, are flexible and can be adapted to individual wishes and circumstances. The concept needs a general categorization (floor, walls, roof).

In our case, we need to create a concept for the attacks on all possible services and divide it into categories, but leave the individual attack methods.

To explain a little more clearly what we are talking about here, we can try to group the services SSH, FTP, and so on. We need to figure out what these services have in common. Then we need to create a structure that will allow us to identify different services using a single pattern.

Analyzing commonalities and creating pattern templates that fit all conceivable cases is not a finished product. It makes these pattern templates grow larger and larger. Therefore, we have created a pattern template for teaching more efficiently teach and explain the concept behind the attacks.

The Concept of Attacks



(Destination) to be achieved. Therefore, the category of Privileges is necessary to control information p

Source

We can generalize Source as a source of information used for the specific task of a process. There are ma
information to a process. The graphic shows some of the most common examples of how information is p

Information Source	Description
Code	This means that the already executed program code results are used as a source of information. The of a program.
Libraries	A library is a collection of program resources, including configuration data, documentation, help data and subroutines, classes, values, or type specifications.
Config	Configurations are usually static or prescribed values that determine how the process processes info
APIs	The application programming interface (API) is mainly used as the interface of programs for retrieving
User Input	If a program has a function that allows the user to enter specific values used to process the informati entry of information by a person.

The source is, therefore, the source that is exploited for vulnerabilities. It does not matter which protocol i
injections can be manipulated manually, as can buffer overflows. The source for this can therefore be cate
closer look at the pattern template based on one of the latest critical vulnerabilities that most of us have h

Log4j

A great example is the critical Log4j vulnerability (CVE-2021-44228) which was published at the end of 20
Library used to log application messages in Java and other programming languages. This library contain
other programming languages can integrate. For this purpose, information is documented, similar to a log
the documentation can be configured extensively. As a result, it has become a standard within many oper
software products. In this example, an attacker can manipulate the HTTP User-Agent header and insert a
intended for the Log4j library. Accordingly, not the actual User-Agent header, such as Mozilla 5.0, is pro

Processes

Log4j

The process of Log4j is to log the User-Agent as a string using a function and store it in the designated location. The process is the misinterpretation of the string, which leads to the execution of a request instead of logging. To go further into this function, we need to talk about privileges.

Privileges

Privileges are present in any system that controls processes. These serve as a type of permission that determines what actions can be performed on the system. In simple terms, it can be compared to a bus ticket. If we use a ticket to enter a region, we will be able to use the bus, and otherwise, we will not. These privileges (or figuratively speaking, tickets) are for different means of transport, such as planes, trains, boats, and others. In computer systems, these privileges represent a segmentation of actions for which different permissions, controlled by the system, are needed. Therefore, the system checks this categorization when a process needs to fulfill its task. If the process satisfies these privileges and controls, the action requested is performed. We can divide these privileges into the following areas:

Privileges	Description
System	These privileges are the highest privileges that can be obtained, which allow any system modification. In Windows, it is called SYSTEM , and in Linux, it is called root .
User	User privileges are permissions that have been assigned to a specific user. For security reasons, separate user accounts are created during the installation of Linux distributions.
Groups	Groups are a categorization of at least one user who has certain permissions to perform specific actions.
Policies	Policies determine the execution of application-specific commands, which can also apply to individual or group users.
Rules	Rules are the permissions to perform actions handled from within the applications themselves.

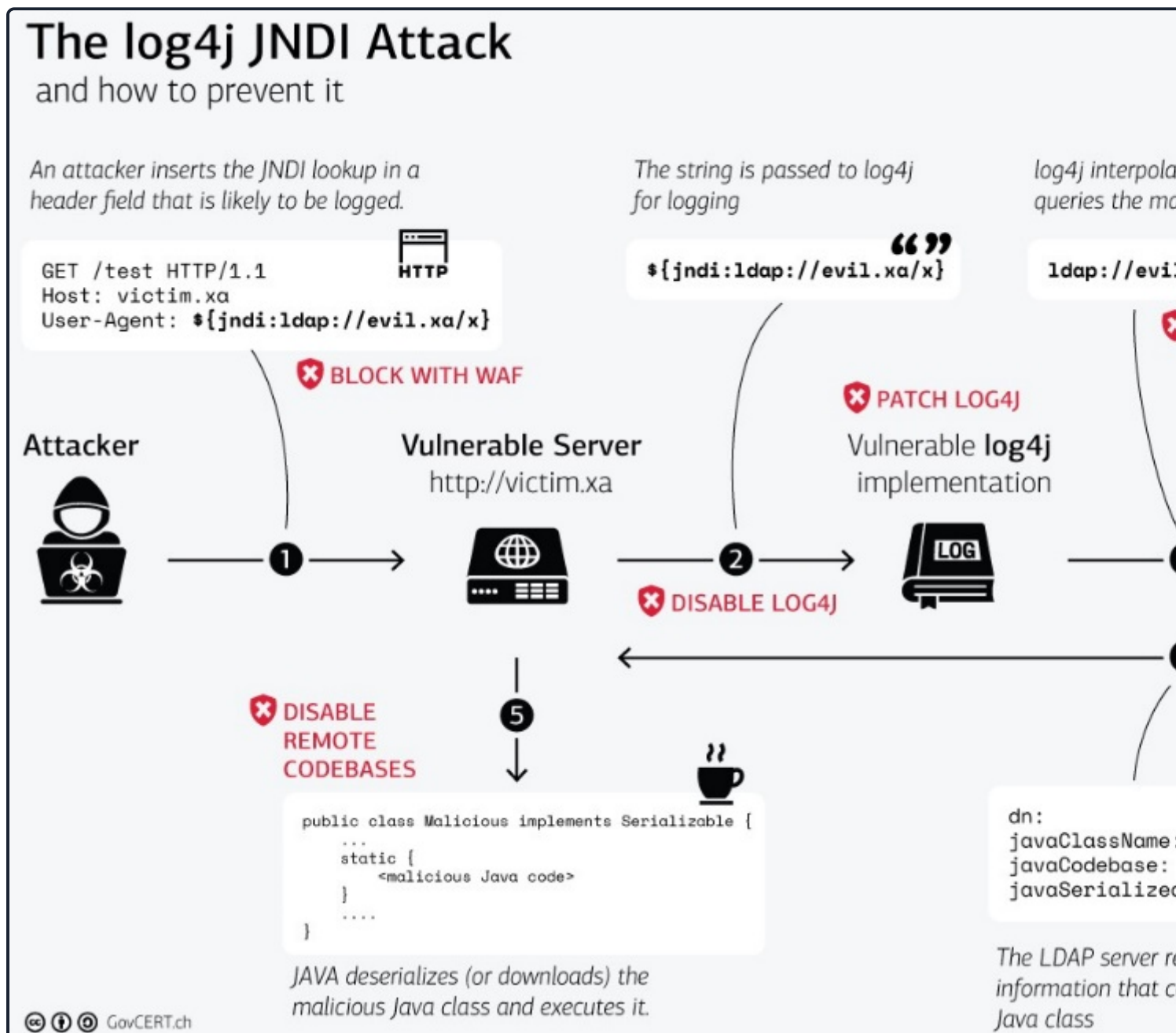
Log4j

What made the Log4j vulnerability so dangerous was the **Privileges** that the implementation brought. Log4j is so sensitive because they can contain data about the service, the system itself, or even customers. Therefore, it can access locations that no regular user should be able to access. Accordingly, most applications with the Log4j implementation have the privileges of an administrator. The process itself exploited the library by manipulating the User-Agent so that it could access the source and led to the execution of user-supplied code.

Log4j

The misinterpretation of the User-Agent leads to a JNDI lookup which is executed as a command from the privileges and queries a remote server controlled by the attacker, which in our case is the **Destination** in the query requests a Java class created by the attacker and is manipulated for its own purposes. The queried manipulated Java class gets executed in the same process, leading to a remote code execution (RCE) vulnerability.

GovCERT.ch has created an excellent graphical representation of the Log4j vulnerability worth examining



Source: <https://www.govcert.ch/blog/zero-day-exploit-targeting-popular-java-library-log4j/>

This graphic breaks down the Log4j JNDI attack based on the **Concept of Attacks**.

5. After the malicious Java class is retrieved from the attacker's server, it is used as a source for further actions in the following process.
6. Next, the malicious code of the Java class is read in, which in many cases has led to remote access to the system.
7. The malicious code is executed with administrator privileges due to logging permissions.
8. The code leads back over the network to the attacker with the functions that allow the attacker to control the system remotely.

Finally, we see a pattern that we can repeatedly use for our attacks. This pattern template can be used to and debug our own exploits during development and testing. In addition, this pattern template can also be analysis, which allows us to check certain functionality and commands in our code step-by-step. Finally, v about each task's dangers individually.

[< Previous](#)[Next >](#)

+10 Strea

[📄 Cheat Sheet](#)[☰ Resources](#)

Table of Contents

Introduction

Interacting with Common Services

Protocol Specific Attacks

[The Concept of Attacks](#)

Service Misconfigurations

Finding Sensitive Information

FTP

 [Attacking FTP](#)

Latest FTP Vulnerabilities

SMTP

 Attacking Email Services

Latest Email Service Vulnerabilities

Skills Assessment

 Attacking Common Services - Easy

 Attacking Common Services - Medium

 Attacking Common Services - Hard

My Workstation

O F F L I N E

 Start Instance

 / 1 spawns left